

ALG-Tema4-EJs_VoracesResueltos.pdf



Anónimo



Algorítmica



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

QUIERES
CONSEGUIR
15€??

→ TRÁENOS A TU
CRUSH DE APUNTES ♡
ANTES DE QUE
LOS QUEME 🔥

**WUOLAH**

QUIERES
CONSEGUIR
15€??

TRÁENOS A TU
CRUSH DE APUNTES
ANTES DE QUE
LOS QUEME



TEORÍA DE ALGORITMOS RELACIÓN DE PROBLEMAS TEMA 4. ALGORITMOS GREEDY

1. Se tiene un barco de mercancías cuya capacidad de carga es de k toneladas y un conjunto de contenedores $c_1, \dots, c_n$, cuyos pesos respectivos (expresados también en toneladas) son $p_1, \dots, p_n$. Se sabe que la capacidad del barco es inferior a la suma total de los pesos de los contenedores.

- Diseñar un algoritmo que maximice el número de contenedores cargados.
- Diseñar un algoritmo para maximizar el número de toneladas cargadas.

Indicar las técnicas de diseño utilizadas en ambos problemas y analizar su complejidad.

2. Sean n programas $P_1, \dots, P_n$ que hay que almacenar en un disco. El programa P_i requiere S_i Kilo bites de espacio y la capacidad del disco es D Kilo bites, donde $D < \sum S_i$. Resolver las siguientes cuestiones:

- Se desea maximizar el número de programas almacenados en el disco. Construir un algoritmo greedy para resolver el problema (existe un algoritmo que da una solución óptima).
- Se desea utilizar la mayor capacidad posible del disco. Demostrar que podemos utilizar un algoritmo greedy que seleccione los programas por orden no creciente de S_i para obtener la solución exacta o dar un contraejemplo en caso contrario.

3. Tenemos que ejecutar un conjunto de n tareas, cada una de las cuales requiere un tiempo unitario. En un instante $T = 1, 2, \dots$ podemos ejecutar únicamente una tarea. La tarea i produce unos beneficios g_i ($g_i > 0$) sólo en el caso en el que sea ejecutada en un instante anterior o igual a d_i . Utilizando la técnica greedy encontrar un algoritmo que nos permita seleccionar el conjunto de tareas a realizar de forma que nos aseguremos que tenemos la mayor ganancia posible.

Resolver el problema utilizando la siguiente instancia:

i	1	2	3	4
g_i	50	10	15	30
d_i	2	1	2	1

4. Consideremos el grafo no-dirigido $G = (V, E)$. Un conjunto de nodos U se dice un cubrimiento de G si U es un subconjunto de V tal que cada arista en E es incidente al menos en un vértice de U . Un conjunto de nodos es un cubrimiento minimal de G si tiene el mínimo número posible de nodos y verifica esta propiedad. El grado de incidencia de un vértice es el número de aristas que inciden en él.

- Dar un algoritmo greedy para este problema.
- Indicar si este algoritmo genera un conjunto de vértices minimal para cualquier grafo si se utiliza como función de selección el máximo grado de incidencia sobre los nodos no seleccionados.

NOTA: La forma de resolver estos problemas es pensando en posibles funciones de selección y factibilidad, pensar en posibles contraejemplos que hagan que el algoritmo no de una solución óptima y finalmente quedarse con la opción más lógica dando alguna razón (incluso aunque ello no asegure una solución óptima/correcta, salvo que se permita usar otro tipo de técnica que si la dé).

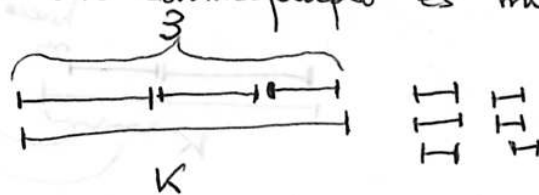


si
consigues
que suba
apuntes, te
llevaré 15€ +
5 Wuolah
Coins para
los sorteos

⑤ Ej. 1. de la relación.

- a)
- Lista de candidatos C : el conjunto de los contenedores
 - Lista de seleccionados S : solución parcial con los contenedores seleccionados hasta el momento
 - Función de factibilidad: Si $\text{peso}(S \cup \{x\}) \leq K$ entonces $S \cup \{x\}$ es factible.
 - Función solución: Cuando no quedan elementos/candidatos factibles.
 - Función objetivo: n° de elementos en S .
 - Función selección: Hay dos posibilidades, coger el siguiente elemento de menor peso o coger el siguiente de mayor peso.

La segunda parece más lógica porque deja el menor espacio posible para colocar más contenedores. De hecho, encontrar un contraejemplo es muy sencillo,



se ve claramente que caben más

Esogeremos por tanto el de menor peso disponible.

Algoritmo = $\text{Maxcont}(C, S)$

```

S ← ∅
P ← 0
mientras (C ≠ ∅) hacer
    x ← buscar_mínimo(C) // n, n-1, n-2...
    C ← C - {x}
    Si (P + peso(x) ≤ K)
        S ← S ∪ {x}
        P ← P + peso(x)
    }
}
    
```

$\left. \begin{array}{l} \text{Si } (P + \text{peso}(x) \leq K) \\ S \leftarrow S \cup \{x\} \\ P \leftarrow P + \text{peso}(x) \end{array} \right\} O(1) \quad \Rightarrow \quad O(n^2)$

**NO
QUEMES
TUS
APUNTES**



si juegas con fuego
te fuebas

**GANAR
0,25 €**

por cada PDF tuyo subido de calidad

* válido hasta el 3 de junio de 2022 o hasta llegar al
tope de documentos para esta promoción

WUOLAH

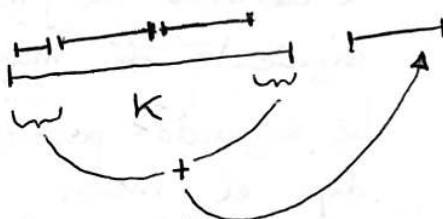
Sin embargo, en este caso se puede realizar una ordenación previa para simplemente ir seleccionando los contenedores mientras quepan. Este algoritmo sería $n \cdot \lg(n) + n \Rightarrow O(n \cdot \lg(n))$.

Queda como ejercicio describir/proponer el algoritmo que usando mergesort o quicksort (éstos no habría que describirlos) dé una solución al problema.

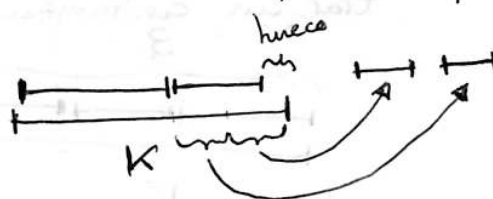
—————

b) De las dos posibilidades anteriormente comentadas, para este problema ninguna nos lleva a un algoritmo correcto mediante el enfoque Greedy. Es fácil encontrar contraejemplos:

1. Escoger el menor disponible:



2. Escoger el mayor disponible:



Este apartado está puesto para que se responda que lo mejor es utilizar una técnica de ramificación y poda, y se describa dicho algoritmo.

—————

② Ej. 2 de la relación.

a) Se resuelve como el apartado a) del ejercicio anterior (es el mismo problema).

b). Es el mismo problema del apartado b) del ejercicio anterior. Bastaría con poner el 2º contra ejemplo.



3 Ej. 3 de la relación.

- Lista de candidatos C : el conjunto de las tareas
- Lista de seleccionados S : las tareas seleccionadas hasta el momento.
- Función solución: cuando no queden más tareas factibles.
- Función objetivo: Beneficios acumulados por las tareas en S . $\text{Beneficio} = \sum_{i \in S} g_i$.
- Función de factibilidad: $(S \cup \{x\})$ es factible si todas las tareas $i \in (S \cup \{x\})$ pueden ser ejecutadas en un instante anterior o igual a d_i .
- Función de selección: La mejor opción es coger las tareas con mayor beneficio. Se pueden encontrar contraejemplos fáciles para otras alternativas lógicas. Por ejemplo, escoger primero aquellas que deban ser ejecutadas antes, contra ejemplo:

i	1	2	3	4	5
g_i	10	5	50		
d_i	2	2	3		

← $\begin{matrix} g_i & 30 \\ d_i & 3 \end{matrix}$ ésta ya no entraría, sin embargo es mejor que las dos primeras.

QUIERES
CONSEGUIR
15€?? →

TRÁENOS A TU
CRUSH DE APUNTES

ANTES DE QUE
LOS QUEME



Algoritmo detallado:

(pg 4)

La parte complicada de este ejercicio está en poder comprobar de manera eficiente si incluir una nueva tarea genera conflicto. Para ello se puede mantener el vector solución S ordenado ^(según d_i), tratando de insertar la nueva tarea en su posición correspondiente. Es fácil comprobar después si alguna de las tareas desplazadas o la propia tarea insertada se encuentra en una posición mayor a su correspondiente d_i , en cuyo caso añadir la tarea no es factible. (Se numera i desde 0).

```
void factible_insertar (datos * S, int & elems, datos x) {
    // datos es una estructura con los valores  $d_i$  y  $g_i$ 
    int factible = 1, i, pos;
    for (i = elems; i > 0 && x.d < S[i-1].d && factible; i--)
        if (S[i-1] > x)
            factible = 0;
    pos = i;
    if (factible && x.d <= pos) { // comprobación factibilidad
        for (i = elems; i > pos; i--) { // inserción
            S[i] = S[i-1];
        }
        S[pos] = x;
        elems++;
    }
    return;
}
```

```
void MaxBeneficio (datos * G, int n, datos * S, int & elems) {
    int i;
    OrdenarPorBeneficios (G, n);
    elems = 0;
}
```



si
consigues
que suba
apuntes, te
llevas 15€ +
5 Wuolah
Coins para
los sorteos

* for ($i=0; i < n; i++$)
 factible-insertar ($S, elems, G[i]$);
 return;

}

Instancia:

i	0	1	2	3
g_i	50	40	15	30
d_i	1	0	1	0

Se adaptan los valores para que estén desde 0.

Ordenación G :

$G = \begin{matrix} g \\ d \end{matrix} \begin{bmatrix} 50 & 30 & 15 & 10 \\ 1 & 0 & 1 & 0 \end{bmatrix}$

Iteración 0:

$S = \begin{matrix} g \\ d \end{matrix} \begin{bmatrix} 50 \\ 1 \end{bmatrix}$
$elems = 1$

Iteración 1:

$S = \begin{bmatrix} 30 & 50 \\ 0 & 1 \end{bmatrix}$
$elems = 2$

Iteración 2 y 3: ningún otro elemento puede ser insertado

$S = \begin{bmatrix} 30 & 50 \\ 0 & 1 \end{bmatrix}$
$elems = 2$

Solución óptima con beneficio 80.

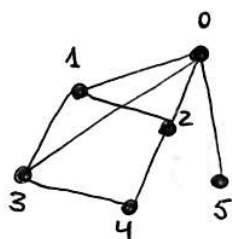
Para ver por qué funciona habría que demostrarlo:

1. Demostrar que la primera elección es la correcta.
2. Demostrar que existen subestructuras óptimas.

Se deja como ejercicio por resolver.

④ Ej. 4 de la relación.

- b) Se puede encontrar un contraejemplo en el que no funciona al considerar el máximo grado de incidencia sobre los nodos no seleccionados.
- Contraejemplos:



incidencias =

4	3	3	3	2	1
---	---	---	---	---	---

0 1 2 3 4 5
↑
vértices

cojiéndolos en este orden (de mayor a menor grado),
 la solución sería,

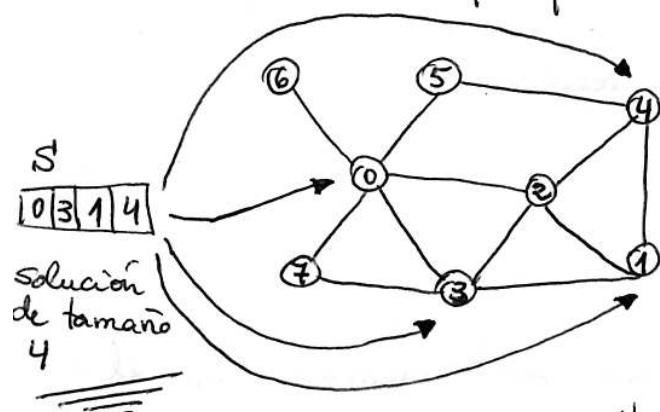
$S = \boxed{0 \mid 1 \mid 2 \mid 3}$ con 4 vértices.

Sin embargo $S = \boxed{0 \mid 3 \mid 2}$ también es solución con sólo 3 vértices.

El algoritmo así no es correcto.

- a) Tal cual están planteadas las preguntas a) y b), podemos proponer un algoritmo que seleccione los vértices por su grado de incidencia, sabiendo que es una buena alternativa pero que no es un algoritmo correcto.
 Hay más alternativas. Por ejemplo:

- 1) Seleccionar el vértice al que más aristas le queden sin cubrir. Implica actualizar el número de aristas cada vez que se selecciona un vértice. Sin embargo, aunque funciona para el caso anterior, sigue existiendo un contraejemplo:



incidencias:

5	3	4	4	3	2	1	2
---	---	---	---	---	---	---	---

 libres:

5	3	4	4	3	2	1	2
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7

iteración 0:
 $S: \boxed{0}$

libres:

0	3	3	3	3	1	0	1
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7

iteración 1:
 $S: \boxed{0 \mid 1}$
 libres:

0	0	2	2	2	1	0	1
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7

iteración 2:
 $S: \boxed{0 \mid 1 \mid 2}$
 libres:

0	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7



iteración 3:

S:

0	1	2	3
---	---	---	---

libres:

0	0	0	0	1	1	0	0
0	1	2	3	4	5	6	7

iteración 4:

S:

0	1	2	3	4
---	---	---	---	---

libres:

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Solución de tamaño
5 !!!

2) También se pueden combinar ambas medidas

- Escoger el que cubra un mayor número de aristas libres y si hay varios con el mismo número, escoger el de mayor grado de incidencia. Este funciona en los dos ejemplos propuestos y no resulta fácil encontrar un contraejemplo. Sin embargo también existe un contraejemplo.

Por simplicidad, y puesto que sería correcto responder así en un examen, vamos a resolverlo respecto a únicamente el grado de incidencia.

- lista de candidatos G : el conjunto de los vértices.
- lista de seleccionados S : los vértices seleccionados hasta el momento.
- Función objetivo: tamaño de S .
- Función de factibilidad: que el vértice seleccionado tenga aristas por abrir.
- Función de selección: el de mayor grado de incidencia.

Puesto que el grado de incidencia es fijo se puede realizar una ordenación al principio. Se supone que G es un vector cuyos elementos tienen dos campos,

QUIERES
CONSEGUIR
15€??

TRÁENOS A TU
CRUSH DE APUNTES

ANTES DE QUE
LOS QUEME



grado y vértice (tipo datos).

PS 8

```
void cubre (datos * G, int n, int * S, int & elems,  
            tipo-grafo G) {  
    int i;
```

```
    OrdenarPorGrado (G, n);  
    elems = 0;
```

```
    for (i = 0; i < n; i++)
```

```
        if (quedan-arcos (G[i])) {  
            S[elems] = G[i].vertice;  
            ActualizarGrados (G, n, i);  
            elems++;  
        }
```

```
    return;
```

```
}
```

equivalente a
(G[i].grado != 0)

esta operación no
cambia el orden de los
elementos en G.

se recorren todos
los elementos desde
i en adelante y
si su grado es
distinto de cero y
están conectados con
G[i].vertice se decre-
mentan sus grados
en 1.



si
consigues
que suba
apuntes, te
llevas 15€ +
5 Wuolah
Coins para
los sorteos